

Summary

OOP is a useful feature for organizing your code. Classes allow you to group together data and code into new data types. You can also create objects from these classes by calling their constructors (the class's name called as a function), which in turn, calls the class's `__init__()` method. Methods are functions associated with objects, and attributes are variables associated with objects. All methods have a `self` parameter as their first parameter, which is assigned the object when the method is called. This allows the methods to read or set the object's attributes and call its methods.

Although Python doesn't allow you to specify private or public access for attributes, it does have a convention of using an underscore prefix for any methods or attributes that should only be called or accessed from the class's own methods. By following this convention, you can avoid misusing the class and setting it into an invalid state that could cause bugs. Calling `type(obj)` will return the `obj` type's class object. Class objects have a `__qualname__` attribute, which contains a string with a human-readable form of the class's name.

At this point, you might be thinking, why we should bother using classes, attributes, and methods when we could do the same task with functions? OOP is a useful way to organize your code into more than just a `.py` file with 100 functions in it. By breaking up your program into several well-designed classes, you can focus on each class separately.

OOP is an approach that focuses on data structures and the methods to handle those data structures. This approach isn't mandatory for every program, and it's certainly possible to overuse OOP. But OOP provides opportunities to use many advanced features that we'll explore in the next two chapters. The first of these features is inheritance, which we'll delve into in the next chapter.